

Deployment-Dokumentation

plotproject auf dem Raspberry Pi 5

Containerisierung mit Docker, Nginx und Gunicorn

System: Raspberry Pi 5, Raspberry Pi OS Bookworm (64-bit)

Stack: Django 6 · Gunicorn · Nginx · Docker Compose

Datum: 29. Juni 2026

Inhaltsverzeichnis

1	Überblick	3
1.1	Architektur	3
1.2	Wichtige Designentscheidungen	3
2	Schritt 1: Docker installieren	3
2.1	Voraussetzung: 64-Bit-Betriebssystem	3
2.2	System aktualisieren	4
2.3	Docker installieren	4
2.4	Installation prüfen	4
3	Schritt 2: Projekt auf den Pi bringen	4
4	Schritt 3: Konfigurationsdateien	4
4.1	Dockerfile	5
4.2	docker-compose.yml	5
4.3	nginx.conf	6
4.4	.dockerignore	7
5	Schritt 4: Django-Einstellungen anpassen	7
5.1	Secret Key auslagern	7
5.2	settings.py anpassen	7
6	Ersteinrichtung	8
6.1	Datenbank vorbereiten	8
6.2	Migrationen ausführen	8
6.3	Static Files einsammeln	8
6.4	Benutzer anlegen	8
7	Container starten	9
8	Erweiterung 1: USB-Gerät (/dev/ttyACM0) einbinden	9
8.1	Problem: Gerät beim Start nicht vorhanden	9
8.1.1	Schritt 1: Compose-Version prüfen	9
8.1.2	Schritt 2a: Compose aktualisieren (falls Version < 2.23)	9
8.1.3	Schritt 2b: docker-compose.yml anpassen	10
8.2	Stabiler Geräteiname per udev-Regel (optional)	10
9	Erweiterung 2: Autostart beim Pi-Boot	11
9.1	Service-Datei anlegen	11
9.2	Service aktivieren	11
9.3	Verhalten testen	12
10	Entwicklungsworkflow	12
10.1	Alias für Docker-Compose-Befehle	12
10.2	Django-Befehle im Container	12

10.3	Getrennte Settings für Entwicklung und Produktion	12
10.4	GitHub-Authentifizierung per SSH-Key	13
10.5	Welche Dateien gehören ins Repository?	13
11	Erweiterung 3: HTTPS mit eigener Zertifizierungsstelle	14
11.1	Konzept	14
11.2	Schritt 1: CA erstellen	14
11.3	Schritt 2: Serverzertifikat erstellen	14
11.4	Schritt 3: Nginx für HTTPS konfigurieren	15
11.5	Schritt 4: Django für den Parallelbetrieb anpassen	16
11.6	Schritt 5: CA-Zertifikat auf Clients installieren	17
12	Zusammenfassung	18

1 Überblick

Dieses Dokument beschreibt die vollständige Einrichtung des `plotproject`-Webservers auf einem Raspberry Pi 5 mithilfe von Docker. Das Projekt ist eine Django-Webanwendung zur Echtzeit-Temperaturüberwachung mehrerer LoRa-Sensorknoten.

1.1 Architektur

Der Gesamtstack besteht aus drei unabhängigen Docker-Containern, die über Docker Compose orchestriert werden:

Container	Image	Aufgabe
<code>web</code>	eigenes Dockerfile	Django + Gunicorn (Port 8000, intern)
<code>nginx</code>	<code>nginx:stable</code>	Reverse-Proxy, Static Files (Port 80)
<code>lora-rx</code>	eigenes Dockerfile	LoRa-Empfänger-Prozess (<code>lora_receiver.py</code>)

Die SQLite-Datenbank (`db.sqlite3`) liegt auf dem Host-Dateisystem und wird in beide Python-Container eingebunden (Bind Mount). Dadurch teilen `web` und `lora-rx` dieselbe Datenbank, ohne über ein Netzwerk kommunizieren zu müssen.

1.2 Wichtige Designentscheidungen

- **Ein Image, zwei Container:** `web` und `lora-rx` bauen auf demselben Dockerfile auf. Der Unterschied liegt nur im Startbefehl (`CMD` vs. `command`: in Compose).
- **Kein docker-Gruppe-Eintrag:** Alle Docker-Befehle erfordern `sudo`. Bewusste Entscheidung – ein physisch zugänglicher Kiosk-Pi soll nicht ohne Passwort administriert werden können.
- **HTTP statt HTTPS:** Das System ist ausschließlich im lokalen Netzwerk erreichbar; auf sensible Daten wird nicht zugegriffen. HTTPS wurde daher bewusst weggelassen.
- **SQLite mit WAL-Modus:** Gleichzeitiger Lese-/Schreibzugriff zweier Prozesse auf dieselbe Datei wird durch `PRAGMA journal_mode=WAL` abgesichert (bereits im `lora_receiver.py` implementiert).

2 Schritt 1: Docker installieren

2.1 Voraussetzung: 64-Bit-Betriebssystem

Docker setzt auf dem Raspberry Pi ein 64-Bit-Betriebssystem voraus. Die Architektur lässt sich wie folgt prüfen:

```
uname -m
# Erwartete Ausgabe: aarch64
```

2.2 System aktualisieren

```
sudo apt-get update && sudo apt-get upgrade -y
```

2.3 Docker installieren

Docker stellt ein offizielles Convenience-Skript bereit, das Architektur und Betriebssystem automatisch erkennt und die passenden Pakete installiert:

```
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh
```

Das Skript installiert folgende Komponenten: `docker-ce`, `docker-ce-cli`, `containerd.io`, `docker-buildx-plugin` sowie `docker-compose-plugin`.

2.4 Installation prüfen

```
sudo docker --version
sudo docker compose version
sudo docker run hello-world
```

Ein erfolgreich abgeschlossener `hello-world`-Lauf bestätigt, dass Docker Engine und Compose-Plugin korrekt installiert sind.

Hinweis: Der Benutzer wurde bewusst *nicht* zur `docker`-Gruppe hinzugefügt. Sämtliche Docker-Befehle erfordern daher `sudo` und damit eine Passworтеingabe.

3 Schritt 2: Projekt auf den Pi bringen

Das Projekt liegt als öffentliches Repository auf GitHub und wird direkt auf dem Pi geklont:

```
cd ~/DockerFiles
git clone https://github.com/<username>/<repo>.git
```

Nach dem Klonen sollte der Projektordner folgende Struktur aufweisen:

```
ls ~/DockerFiles/plotproject
# manage.py lora_receiver.py mysite/ plot/ requirements.txt ...
```

4 Schritt 3: Konfigurationsdateien

Im Projektordner werden vier neue Dateien angelegt.

4.1 Dockerfile

Das Dockerfile beschreibt das gemeinsame Basis-Image für `web` und `lora-rx`:

Listing 1: Dockerfile

```
FROM python:3.12-slim

ENV PYTHONUNBUFFERED=1 \
    PYTHONDONTWRITEBYTECODE=1

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

CMD ["gunicorn", "mysite.wsgi:application",
    "--bind", "0.0.0.0:8000", "--workers", "2"]
```

Anmerkungen:

- `python:3.12-slim` – minimales Basis-Image, funktioniert automatisch auf ARM64.
- `PYTHONUNBUFFERED` – Logs erscheinen sofort in `docker logs`, ohne Puffer-Verzögerung.
- `requirements.txt` wird vor dem restlichen Code kopiert, damit Docker den `pip install`-Schritt beim nächsten Build aus dem Cache verwenden kann, solange sich die Abhängigkeiten nicht geändert haben.
- `collectstatic` wird *nicht* im Dockerfile ausgeführt, da beim Build-Zeitpunkt die `.env`-Datei nicht verfügbar ist (und damit `SECRET_KEY` fehlen würde). Stattdessen wird `collectstatic` einmalig manuell ausgeführt (siehe Abschnitt 6).

4.2 docker-compose.yml

Listing 2: docker-compose.yml

```
services:

  web:
    build: .
    expose:
      - "8000"
    volumes:
      - ./db.sqlite3:/app/db.sqlite3
      - ./staticfiles:/app/staticfiles
    env_file:
      - .env
    restart: unless-stopped
```

```
lora-rx:
  build: .
  command: python lora_receiver.py
  volumes:
    - ./db.sqlite3:/app/db.sqlite3
  restart: unless-stopped

nginx:
  image: nginx:stable
  ports:
    - "80:80"
  volumes:
    - ./nginx.conf:/etc/nginx/conf.d/default.conf:ro
    - ./staticfiles:/app/staticfiles:ro
  depends_on:
    - web
  restart: unless-stopped
```

Anmerkungen:

- **expose** vs. **ports**: **expose** macht Port 8000 nur für andere Container im selben Compose-Netzwerk sichtbar. Nur Nginx hat **ports** und ist damit von außen erreichbar.
- **db.sqlite3** wird als Bind Mount in beide Python-Container eingebunden – so teilen sie dieselbe Datenbankdatei über das Host-Dateisystem.
- **restart: unless-stopped** sorgt dafür, dass Container nach einem Absturz automatisch neu gestartet werden.

4.3 nginx.conf

Listing 3: nginx.conf

```
server {
    listen 80;

    location /static/ {
        alias /app/staticfiles/;
    }

    location / {
        proxy_pass http://web:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

Anmerkungen:

- Statische Dateien werden direkt von Nginx ausgeliefert, ohne Django zu involvieren –

das ist deutlich performanter.

- `http://web:8000`: Docker löst `web` automatisch zur internen IP des gleichnamigen Containers auf.
- `proxy_set_header` reicht die originale Client-IP und den Host-Header durch, damit Django `ALLOWED_HOSTS` korrekt prüfen kann.

4.4 .dockerignore

Listing 4: .dockerignore

```
db.sqlite3
staticfiles/
venv/
.env
__pycache__/*
*.pyc
*.pyo
.git/
```

Verhindert, dass unnötige oder sicherheitskritische Dateien ins Image kopiert werden – insbesondere `.env` (enthält den Secret Key) und `db.sqlite3` (wird zur Laufzeit eingebunden, nicht gebraucht).

Schritt 4: Django-Einstellungen anpassen

5.1 Secret Key auslagern

Der im Repository hinterlegte `SECRET_KEY` gilt als kompromittiert und muss ersetzt werden. Ein neuer Schlüssel wird generiert und in einer `.env`-Datei gespeichert, die niemals ins Repository gepusht wird.

```
# Neuen Key generieren
openssl rand -base64 50

# .env-Datei anlegen
nano ~/DockerFiles/plotproject/.env
# Inhalt: SECRET_KEY=<generierter-schluessel>

# Sicherstellen, dass .env nicht eingecheckt wird
echo ".env" >> ~/DockerFiles/plotproject/.gitignore
```

5.2 settings.py anpassen

Drei Stellen in `mysite/settings.py` müssen geändert werden:

Listing 5: `mysite/settings.py` – relevante Änderungen


```
import os # am Anfang der Datei ergaenzen

# Secret Key aus Umgebungsvariable lesen
SECRET_KEY = os.environ.get('SECRET_KEY')

# Debug-Modus deaktivieren
DEBUG = False

# Erlaubte Hosts explizit setzen
ALLOWED_HOSTS = ['192.168.2.65', 'localhost', '127.0.0.1']
```

Hinweis zu ALLOWED_HOSTS: Mit `DEBUG = False` lehnt Django alle Requests ab, deren Host-Header nicht in dieser Liste steht. Die Pi-IP muss daher eingetragen sein.

6 Ersteinrichtung

Vor dem ersten Start müssen drei einmalige Schritte ausgeführt werden.

6.1 Datenbank vorbereiten

Docker erstellt beim Bind Mount automatisch einen Ordner, wenn die Zielfeile nicht existiert. Um das zu verhindern, wird die leere Datei vorab angelegt:

```
touch ~/DockerFiles/plotproject/db.sqlite3
mkdir -p ~/DockerFiles/plotproject/staticfiles
```

6.2 Migrationen ausführen

```
sudo docker compose -f ~/DockerFiles/plotproject/docker-compose.yml \
    run --rm web python manage.py migrate
```

6.3 Static Files einsammeln

```
sudo docker compose -f ~/DockerFiles/plotproject/docker-compose.yml \
    run --rm web python manage.py collectstatic --noinput
```

6.4 Benutzer anlegen

Da die Datenbank neu erstellt wurde, muss der Benutzer für das Login-System erneut angelegt werden:

```
sudo docker compose -f ~/DockerFiles/plotproject/docker-compose.yml \
    run --rm web python manage.py shell -c "
from django.contrib.auth.models import User
User.objects.create_user('shared', password='<password>')
```

"

7 Container starten

```
# Alle Container im Hintergrund starten
sudo docker compose -f ~/DockerFiles/plotproject/docker-compose.yml up -d

# Status pruefen
sudo docker compose -f ~/DockerFiles/plotproject/docker-compose.yml ps

# Logs beobachten
sudo docker compose -f ~/DockerFiles/plotproject/docker-compose.yml logs
```

Das Dashboard ist danach unter <http://192.168.2.65> erreichbar.

Nützliche Befehle im laufenden Betrieb:

Aktion	Befehl
Alle Container stoppen	<code>sudo docker compose ... down</code>
Logs eines Containers	<code>sudo docker compose ... logs web</code>
Container neu starten	<code>sudo docker compose ... restart web</code>
Neu bauen und starten	<code>sudo docker compose ... up -build -d</code>

8 Erweiterung 1: USB-Gerät (/dev/ttyACM0) einbinden

Das LoRa-Empfangsmodul ist per USB am Pi angebunden und erscheint als `/dev/ttyACM0`. Damit `lora-rx` im echten Betrieb (nicht im Dummy-Modus) darauf zugreifen kann, muss das Gerät in den Container durchgereicht werden.

8.1 Problem: Gerät beim Start nicht vorhanden

Wenn `/dev/ttyACM0` beim Starten von Docker Compose nicht existiert, bricht Docker mit einem Fehler ab. Die `required: false`-Option (die dieses Verhalten unterdrückt) ist erst ab Docker Compose v2.23 verfügbar.

8.1.1 Schritt 1: Compose-Version prüfen

```
sudo docker compose version
```

8.1.2 Schritt 2a: Compose aktualisieren (falls Version < 2.23)

```
# Aktuellste Version des Compose-Plugins installieren
sudo apt-get update
```

```
sudo apt-get install --only-upgrade docker-compose-plugin
```

8.1.3 Schritt 2b: docker-compose.yml anpassen

Mit Compose $\geq$ 2.23 kann das Gerät als optional markiert werden:

Listing 6: lora-rx mit optionalem USB-Device

```
lora-rx:
  build: .
  command: python lora_receiver.py
  devices:
    - source: "/dev/ttyACM0"
      target: "/dev/ttyACM0"
      required: false
  volumes:
    - ./db.sqlite3:/app/db.sqlite3
  restart: unless-stopped
```

Mit `required: false` startet der Container auch ohne angestecktes Gerät – dann greift `lora_receiver.py` automatisch auf den Dummy-Modus zurück.

8.2 Stabiler Geräteiname per udev-Regel (optional)

Der Geräteiname `ttyACM0` kann sich ändern, wenn andere USB-Geräte angeschlossen sind. Eine udev-Regel fixiert das Modul dauerhaft auf einen festen Namen:

```
# Vendor- und Product-ID des Moduls ermitteln
lsusb
# Beispielausgabe: Bus 001 Device 003: ID 10c4:ea60 ...

# udev-Regel anlegen
sudo nano /etc/udev/rules.d/99-lora.rules
```

Listing 7: /etc/udev/rules.d/99-lora.rules

```
SUBSYSTEM=="tty", ATTRS{idVendor}=="10c4", ATTRS{idProduct}=="
    ea60",
SYMLINK+="lora_receiver"
```

```
# Regel aktivieren
sudo udevadm control --reload-rules
sudo udevadm trigger
```

Das Modul ist danach zusätzlich unter `/dev/lora_receiver` erreichbar und in `docker-compose.yml` sowie `lora_receiver.py` kann dieser stabile Name verwendet werden.

9 Erweiterung 2: Autostart beim Pi-Boot

Aktuell müssen die Container nach einem Neustart manuell gestartet werden. Ein systemd-Service übernimmt das automatisch.

9.1 Service-Datei anlegen

```
sudo nano /etc/systemd/system/plotproject.service
```

Listing 8: /etc/systemd/system/plotproject.service

```
[Unit]
Description=plotproject Docker Compose Stack
Requires=docker.service
After=docker.service network-online.target
Wants=network-online.target

[Service]
Type=oneshot
RemainAfterExit=yes
WorkingDirectory=/home/lorabase/DockerFiles/plotproject
ExecStart=/usr/bin/docker compose up -d
ExecStop=/usr/bin/docker compose down
TimeoutStartSec=120

[Install]
WantedBy=multi-user.target
```

Anmerkungen:

- `Requires=docker.service` – stellt sicher, dass Docker läuft, bevor der Stack gestartet wird.
- `After=network-online.target` – wartet auf eine Netzwerkverbindung (wichtig für das Ziehen von Images).
- `Type=oneshot` mit `RemainAfterExit=yes` – der Service gilt als aktiv, solange die Container laufen, auch wenn `docker compose up -d` selbst bereits beendet wurde.
- `TimeoutStartSec=120` – gibt Docker ausreichend Zeit, Images zu laden und Container zu starten.

9.2 Service aktivieren

```
# Systemd neu laden
sudo systemctl daemon-reload

# Service beim Boot aktivieren
sudo systemctl enable plotproject.service
```

```
# Service sofort starten (zum Testen)
sudo systemctl start plotproject.service

# Status pruefen
sudo systemctl status plotproject.service
```

9.3 Verhalten testen

```
# Pi neu starten
sudo reboot

# Nach dem Neustart: Container-Status pruefen
sudo docker compose -f ~/DockerFiles/plotproject/docker-compose.yml ps
```

Alle drei Container sollten ohne manuellen Eingriff hochgefahren sein.

10 Entwicklungsworkflow

10.1 Alias für Docker-Compose-Befehle

Da alle Docker-Befehle den langen Pfad zur `docker-compose.yml` sowie `sudo` benötigen, empfiehlt sich ein Alias in der `~/ .bashrc`:

```
echo "alias dc='sudo docker compose -f ~/DockerFiles/plotproject/docker-
compose.yml'" \
>> ~/.bashrc
source ~/.bashrc
```

Danach genügt z. B. `dc up -d`, `dc ps` oder `dc logs`.

10.2 Django-Befehle im Container

Solange Docker verwendet wird, laufen alle Django-Verwaltungsbefehle im Container. Bei *laufenden* Containern ist `exec` effizienter als `run`, da kein neuer Container gestartet wird:

```
dc exec web python manage.py migrate
dc exec web python manage.py collectstatic --noinput
```

Migrationen sind nur nach Modelländerungen nötig – im Normalbetrieb selten.

10.3 Getrennte Settings für Entwicklung und Produktion

Damit Entwicklungs- und Produktionseinstellungen nicht ständig manuell umgeschaltet werden müssen, wird eine separate `settings_dev.py` angelegt, die alle Werte aus `settings.py` erbt und nur die für die Entwicklung relevanten überschreibt:

Listing 9: mysite/settings_dev.py

```

from .settings import *

DEBUG = True
SECRET_KEY = 'dev-only-insecure-key-nicht-fuer-produktion'
ALLOWED_HOSTS = ['*']

# Schnellere Passwort-Hashes beim Testen
PASSWORD_HASHERS = [
    'django.contrib.auth.hashers.MD5PasswordHasher',
]

```

Diese Datei enthält keine Geheimnisse und darf ins Repository. Der Entwicklungsserver wird damit so gestartet:

```
python manage.py runserver --settings=mysite.settings_dev
```

10.4 GitHub-Authentifizierung per SSH-Key

GitHub unterstützt keine Passwort-Authentifizierung mehr. Auf dem Pi wird daher ein SSH-Key erzeugt und bei GitHub hinterlegt:

```

# Key erzeugen (dreimal Enter, keine Passphrase nötig)
ssh-keygen -t ed25519 -C "lorabase-pi"

# Öffentlichen Key anzeigen und bei GitHub hinterlegen
# (Settings -> SSH and GPG keys -> New SSH key)
cat ~/.ssh/id_ed25519.pub

# Remote-URL des Repos auf SSH umstellen
git remote set-url origin git@github.com:<username>/plotproject.git

```

10.5 Welche Dateien gehören ins Repository?

Datei	Ins Repository?
Dockerfile, docker-compose.yml	ja
nginx.conf, .dockerignore	ja
settings.py, settings_dev.py	ja
certs/ca.crt (öffentliches CA-Zertifikat)	ja
.env (Secret Key)	nein
db.sqlite3	nein
staticfiles/ (generiert)	nein
certs/*.key (private Schlüssel)	nein

11 Erweiterung 3: HTTPS mit eigener Zertifizierungsstelle

Das System soll optional über HTTPS erreichbar sein, ohne HTTPS für alle Clients zu erzwingen. Clients, die das CA-Zertifikat installiert haben, erhalten eine vertrauenswürdige HTTPS-Verbindung; alle übrigen nutzen weiterhin HTTP. Dazu wird eine eigene Zertifizierungsstelle (CA) aufgebaut.

11.1 Konzept

Bestandteil	Funktion
CA-Zertifikat (<code>ca.crt</code>)	wird einmalig auf Clients installiert
CA-Schlüssel (<code>ca.key</code>)	signiert Serverzertifikate, bleibt geheim
Serverzertifikat (<code>server.crt</code>)	von der CA signiert, für Nginx
Serverschlüssel (<code>server.key</code>)	privat, bleibt auf dem Pi

11.2 Schritt 1: CA erstellen

```
mkdir ~/DockerFiles/plotproject/certs
cd ~/DockerFiles/plotproject/certs

# Privaten CA-Schlüssel erzeugen
openssl genrsa -out ca.key 4096

# Selbstsigniertes CA-Zertifikat ausstellen (10 Jahre gueltig)
openssl req -new -x509 -days 3650 \
  -key ca.key \
  -out ca.crt \
  -subj "/CN=LoraBase-CA/O=LoraBase/C=DE"
```

11.3 Schritt 2: Serverzertifikat erstellen

Moderne Browser akzeptieren ein Zertifikat nur, wenn die IP-Adresse als *Subject Alternative Name* (SAN) eingetragen ist. Dafür wird eine kleine Konfigurationsdatei benötigt.

```
# Serverschlüssel erzeugen
openssl genrsa -out server.key 2048

# Signieranfrage (CSR) erstellen
openssl req -new \
  -key server.key \
  -out server.csr \
  -subj "/CN=192.168.2.65/O=LoraBase/C=DE"
```

Listing 10: certs/san.cnf

```
[req]
distinguished_name = req_distinguished_name

[req_distinguished_name]

[v3_req]
subjectAltName = IP:192.168.2.65
```

```
# CSR durch die CA signieren (825 Tage -- Browser-Maximum)
openssl x509 -req -days 825 \
  -in server.csr \
  -CA ca.crt \
  -CAkey ca.key \
  -CAcreateserial \
  -out server.crt \
  -extensions v3_req \
  -extfile san.cnf
```

Hinweis: Das Serverzertifikat ist 825 Tage gültig (Browser akzeptieren keine längere Laufzeit) und muss nach knapp zwei Jahren erneuert werden. Die CA selbst bleibt zehn Jahre gültig.

11.4 Schritt 3: Nginx für HTTPS konfigurieren

Die `nginx.conf` erhält einen zweiten Server-Block für Port 443. Der Port-80-Block bleibt für Clients ohne CA erhalten.

Listing 11: `nginx.conf` – mit HTTPS

```
server {
    listen 80;

    location /static/ {
        alias /app/staticfiles/;
    }

    location / {
        proxy_pass http://web:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}

server {
    listen 443 ssl;

    ssl_certificate      /etc/nginx/certs/server.crt;
    ssl_certificate_key  /etc/nginx/certs/server.key;
```



```

    location /static/ {
        alias /app/staticfiles/;
    }

    location / {
        proxy_pass http://web:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}

```

In der `docker-compose.yml` wird Port 443 geöffnet und das Zertifikatsverzeichnis eingebunden:

Listing 12: `docker-compose.yml` – nginx-Block

```

nginx:
  image: nginx:stable
  ports:
    - "80:80"
    - "443:443"
  volumes:
    - ./nginx.conf:/etc/nginx/conf.d/default.conf:ro
    - ./staticfiles:/app/staticfiles:ro
    - ./certs/server.crt:/etc/nginx/certs/server.crt:ro
    - ./certs/server.key:/etc/nginx/certs/server.key:ro
  depends_on:
    - web
  restart: unless-stopped

```

11.5 Schritt 4: Django für den Parallelbetrieb anpassen

Beim Betrieb hinter einem TLS-terminierenden Proxy entkoppelt Nginx die HTTPS-Verbindung und reicht den Request intern als HTTP an Gunicorn weiter. Django sieht dadurch ein anderes Protokoll, als der Browser meldet – das führt beim Login zu einem CSRF-Fehler (HTTP 403). Folgende Anpassungen in `settings.py` lösen das und stellen den korrekten Parallelbetrieb sicher.

Listing 13: `mysite/settings.py` – HTTPS-relevante Ergänzungen

```

# Vertrauenswuerdige Origins fuer CSRF -- beide Schemata
CSRF_TRUSTED_ORIGINS = [
    'https://192.168.2.65',
    'http://192.168.2.65',
]

# Nginx meldet das urspruengliche Protokoll ueber diesen Header

```

```
SECURE_PROXY_SSL_HEADER = ( 'HTTP_X_FORWARDED_PROTO' , 'https' )
```

Wichtige Fallstricke beim Parallelbetrieb HTTP + HTTPS:

- `SESSION_COOKIE_SECURE` und `CSRF_COOKIE_SECURE` bleiben auf `False`. Auf `True` gesetzt würden Cookies nur noch über HTTPS gesendet – Clients ohne CA könnten sich dann nicht mehr einloggen.
- `SECURE_SSL_REDIRECT` bleibt deaktiviert. Aktiviert würde es alle HTTP-Anfragen zwangsweise auf HTTPS umleiten und damit den gewollten HTTP-Zugang für Clients ohne CA unterbinden.
- Beide `CSRF_TRUSTED_ORIGINS`-Einträge (HTTP und HTTPS) sind nötig, damit der Login über beide Wege funktioniert.

11.6 Schritt 5: CA-Zertifikat auf Clients installieren

Das öffentliche CA-Zertifikat wird per `scp` auf den Client übertragen (SSH muss dafür auf dem Pi aktiv sein):

```
# SSH auf dem Pi aktivieren (falls noch nicht geschehen)
sudo systemctl enable ssh
sudo systemctl start ssh

# Auf dem Windows-Rechner (PowerShell): CA-Zertifikat holen
scp lorabase@192.168.2.65:~/DockerFiles/plotproject/certs/ca.crt \
  C:\Users\Lukas\Desktop\ca.crt
```

Installation unter Windows:

1. Doppelklick auf `ca.crt` → „Zertifikat installieren“
2. „Lokaler Computer“ auswählen
3. „Alle Zertifikate in folgendem Speicher speichern“ → „Durchsuchen“
4. „Vertrauenswürdige Stammzertifizierungsstellen“ auswählen
5. Firefox vollständig neu starten (liest den Zertifikatspeicher neu ein)

Danach ist `https://192.168.2.65` ohne Browserwarnung erreichbar.

12 Zusammenfassung

Schritt	Ergebnis	Status
Docker installieren	Docker Engine + Compose auf Pi	✓
Projekt klonen	Projektcode via <code>git clone</code>	✓
Konfigurationsdateien	Dockerfile, Compose, Nginx, .dockerignore	✓
Django-Einstellungen	DEBUG, ALLOWED_HOSTS, SECRET_KEY	✓
Ersteinrichtung	migrate, collectstatic, User anlegen	✓
Container starten	Dashboard erreichbar unter Port 80	✓
Entwicklungsworkflow	Alias, settings_dev, SSH-Key	✓
HTTPS mit eigener CA	Port 443 zusätzlich zu HTTP	✓
USB-Gerät einbinden	/dev/ttyACM0 optional passthrough	geplant
Autostart beim Boot	systemd-Service	geplant